MEDIUM-3：初识Linux系统

第一境：算者—— 命令行参数与路径

【任务 1.1：不同的路】

1.

- 绝对路径：在Linux中的形式表现为，从根目录（ / ）开始，直到目标文件或目录的完整路径
  如：/home/mark/a.exe
- 相对路径：在Linux中的形式表现为，相对于用户当前所在的工作目录来定位文件或目录的路径
  多用于检索父目录，当前目录，以及工作目录的家目录中的目标文件或目录

个人感觉有点类似于经典物理当中的 “惯性参考系” 和 “相对参考系”

2.

```
cd /var/backups/turing/ #绝对路径
cd ../../../var/backups/turing/ #相对路径
```

【任务 1.2：何处启程】

1.

- argc 表示命令行参数的总个数
- argv[] 是一个字符指针数组，里面存放了所有的命令行参数内容，其中 argv[argc] 固定为 NULL

2.

针对上述命令，argc = 2，argv[0] 指向 cd，argv[1] 指向 /var/backups/turing/
（或../../../var/backups/turing/），argv[2] 指向 NULL

第二境：算王—— 重建 ls 命令与位运算

【任务 2.1：非同寻常之物】

1.

说实话，看到这么多头文件真是让人头疼啊...emm.....

- 比较熟悉的有：<stdio.h>、<stdlib.h>
- 见过几面的有：<string.h>（获取长度的 strlen（） 用过一次）、<time.h>（time()和clock()在五子棋那道题里了解过）
- 完全没见过的有：<dirent.h>、 <sys/stat.h>、<sys/types.h>

2.

- <dirent.h>
  - 关键结构体
    - DIR：代表一个目录流（本质上是一个不透明的结构体，通过目录流，程序可以按顺序遍历目录下的所有条目）

        - struct dirent：表示目录中的一个条目，核心成员包含文件名（d_name）、inode 号（d_ino）以及文件类型标识（d_type）
    - 关键函数
        - opendir()：打开指定路径的目录，并返回一个 DIR* 目录流指针。如果失败则返回 NULL
        - readdir()：从目录流中读取下一个目录项。每次调用，它都会返回一个指向 struct dirent 的指针。当目录被读取完毕或发生错误时，返回 NULL
        - closedir()：关闭目录流并释放相关资源
- <sys/stat.h>
    - 关键结构体
        - struct stat：用于存储文件的详细元数据属性，包括文件大小（st_size）、文件类型与权限（st_mode）、所有者（st_uid）以及各类时间戳（如修改时间 st_mtime）等
    - 关键函数
        - stat()：通过文件路径名来获取文件信息，若该路径是一个符号链接，则返回链接所指向的真实文件的信息
        - lstat()：和 stat 几乎一样，区别在于，若该路径是一个符号链接，则返回符号链接本身的信息而不是指向文件
        - fstat()：通过已经打开的文件描述符（fd）来获取文件信息（和 stat 一样，只是参数从路径变成了 int fd）

（PS：符号链接是一种特殊的文件类型，存储了目标文件的路径信息，类似于C语言中指针的指针）

【任务 2.2：看看权限】

请思考，整数如何表示文件的权限：

整数对应32位（即4个字节），我推测其中固定位的排列分别表示r，w，x的权限信息，某种特殊排列代表有对应权限，其他表示没有

按照题目的意思，每一位都存储着权限信息，我们默认1为拥有权限，那么就可以通过位运算来判断最低九位中每一位是0还是1，是1则计为r/w/x，是0则计为‘-’，值得注意的是字符串perms低位记录的是mode高位上的信息，另外，屏蔽不需要的高位数据只需要将 mode 和 0777 做“&”运算就可以了

```c
// mode即传入的存储权限信息的整数， 如0754（八进制表示）
// 提示：传入的 mode 变量不仅包含权限信息，高位可能还包含文件类型。在进行位运算时，请注意屏蔽（Mask）掉不需要的高位数据。
void parse_permissions(mode_t mode, char *perms) {

    // 权限字符对照表
    char rwx[] = {'r', 'w', 'x'};

    // 请补全代码写在 TO DO 与 END OF TO DO 之间：
    // TO DO
    for(int i = 0;i < 9;i++){
        perms[i] = (((mode & 0777)>>(8 - i)) & 1)?rwx[i%3]:'-';
    }

    // END OF TO DO

    perms[9] = '\0'; // 字符串结尾
}
```

第三境：算尊—— 深入理解 Inode 与文件树

【任务 3.1：万物皆 Inode】

1.

inode（索引节点）中存储的文件信息：

- 文件类型与权限：记录文件的具体类型（如普通文件、目录、符号链接、设备文件等），以及文件的 r、w、x 权限
- 所有者信息：包含文件所属用户的标识符（UID）和所属组的标识符（GID）
- 文件大小：记录该文件所占用的字节数
- 时间戳：包含三个关键时间记录
    - atime：文件最后一次被访问的时间
    - mtime：文件内容最后一次被修改的时间
    - ctime：Inode 本身最后一次被更改的时间
- 硬链接数：记录当前有多少个文件名指向这个 Inode
- 数据块指针：指向文件实际内容在磁盘上存储的数据块位置

总而言之，Inode 包含了文件的 “元数据”，即除了文件名和实际数据内容之外的所有文件信息

2.

额...上面说过了，inode 不包括文件名和实际数据内容

3.

- “.” 代表当前目录本身，而 ”..“ 代表上一级目录（对于一般目录而言，”.“ 指向自己，”..“ 指向它的父目录）
- 然而，在根目录（/）中 ”.“ 和 ”..“ 的 Inode 编号相同，都指向根目录本身

【任务 3.2：寻我来时路】

- 在初步尝试之中，我的思路是利用循环，通过 “..” 来依次逆推Inode，并用字符串数组来储存，直至当前目录的Inode=根目录的Inode，最后倒序输出即可

  然而，我发现自己对各种函数的输入和返回值的类型并不熟悉，导致各种函数并不能嵌合，难以实现构想
- 我让AI写出了大致的思路，并将我的与之对比，发现：

只用提供的几种函数，写出大致思路

既然只能使用 `stat()` 、 `chdir()` 、 `opendir()` 、 `readdir()` 以及 `.` 和 `..` ，那么实现“获取当前工作目录绝对路径”的核心思路是：**自底向上回溯，并在父目录中通过 inode 匹配寻找当前目录的名字。**

以下是具体的实现思路步骤：

1. 准备工作

- 调用 `stat(".", &statbuf)` 获取当前目录的 `inode` （记为 `cur_ino` ）。
- 调用 `stat("/", &statbuf)` 获取根目录 `/` 的 `inode` （记为 `root_ino` ），作为回溯终止的条件。
- 准备一个“栈”（如二维字符数组），用于存储从底向上找到的每一级目录名。

2. 核心回溯循环

当 `cur_ino != root_ino` 时，不断执行以下操作：

- **切换目录**：调用 `chdir("..")` 进入上一级父目录。
- **获取父目录信息**：调用 `stat(".", &statbuf)` 获取当前父目录的 `inode` （记为 `parent_ino` ），更新 `cur_ino = parent_ino` 。
- **在父目录中“寻亲”**：
  - 调用 `opendir(".")` 打开父目录。
  - 使用 `readdir()` 循环读取父目录下的所有条目。
    - **注意**：遇到 `.` 和 `..` 必须跳过（ `continue` ）。
    - 对读取到的每一个正常条目，调用 `stat(条目名, &statbuf)` 。
    - **关键匹配**：如果该条目的 `st_ino` 等于进入父目录前记录的旧 `cur_ino` ，说明这个条目就是我们要找的子目录名！将其存入栈中，并 `break` 结束本次 `readdir` 循环。
    - 最后记得关闭目录流（ `closedir` ）。

3. 拼接并输出绝对路径

- 当循环结束时（即 `cur_ino == root_ino` ），说明已经退到了根目录 `/` 。
- 此时栈中存储了从底向上的所有目录名。
- 首先输出一个 `/` 。
- 然后从栈顶到栈底（逆序）依次打印出目录名，并在相邻的名字之间加上 `/` 分隔符。

💡 核心思想总结

这套思路的精髓在于： **`chdir("..")` 负责物理位置的向上移动，而 `readdir()` 配合 `stat()` 负责在父目录的逻辑层面“找回”我们刚才走过的那个目录的名字。**

AI采用的方法是前半段和我相同，不同的是它通过 opendir() 和 readdir() 函数来在父目录中寻找 inode 匹配的目录名，用 root_ino、cur_ino 以及 parent_ino 来存储目标目录的Inode（这样就避免了返回值无法参与比较运算的问题），并且利用了栈先进后出的特点，完成了逆序输出目录名操作

我首先考虑的是如何实现栈操作，因为Inode不包含目录名，所以只能选择用字符数组，在定义伊始，我对字符数组的大小产生了犹豫

如果我要储存目录名，栈应该怎么写呢

既然你需要存储的是**目录名（字符串）**，那么栈里的数据单元就必须是字符数组。

为了配合你之前"获取绝对路径"的需求，我为你写了一个专门用于存储目录名的**顺序栈**标准模板。它包含了初始化、入栈、出栈和判断是否为空四个核心操作：

1. 定义栈结构体

c 编辑

```c
#define MAX_DEPTH 100    // 假设目录最深嵌套 100 层
#define MAX_NAME_LEN 256  // 假设单个目录名最长 256 个字符

typedef struct {
    char data[MAX_DEPTH][MAX_NAME_LEN]; // 二维数组，每个元素存一个目录名
    int top;                            // 栈顶指针，-1 表示空栈
} DirStack;
```

AI的意思是尽可能大...行吧.....

在完成栈操作之后

就要考虑 get_inode 函数的主体部分了，先声明好root_ino 和 cur_ino，然后进入逆推循环，因为"在父目录中寻找 inode 匹配的目录名" 这个part的内容较多，AI单独写了一个函数：find_name_in_parent

因为 entry->d_name 和 "." 没办法直接比较，就由strcmp函数通过ASCII码来比较（这让我发现，我之前不用root_ino 和 cur_ino的while循环条件的判断语句似乎加上strcmp也能跑）

strcmp是什么函数

`strcmp` 是 C 语言中用于**字符串比较**的标准库函数，全称为 "string compare"。它的作用是对两个字符串进行逐字符比较，以判断它们的大小或是否相等。

在主循环中最重要的当然是while循环，先切换到父目录，然后再在父目录中找到原来的目录名，压入栈中并更新当前Inode为父目录的Inode

输出尤其要注意Linux的固有格式，isEmpty函数不仅充当了循环条件并且保证了格式的准确

在完善代码的过程中，偶尔会出现一些小插曲，比如stack是用栈还是栈指针，buf的声明等等，在此就不一 一展开了

完全体代码如下，其中注释绝大多数是我手敲的：

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include <dirent.h>

#define MAX_DEPTH 100       // 假设目录最深嵌套 100 层
#define MAX_NAME_LEN 256    // 假设单个目录名最长 256 个字符

typedef struct {
    char data[MAX_DEPTH][MAX_NAME_LEN]; // 二维数组，每个元素存一个目录名
    int top;                            // 栈顶指针，-1 表示空栈
} DirStack;

```

```c
ino_t get_inode(const char *path) {
    struct stat statbuf;
    //获取当前目录信息
    if (stat(path, &statbuf) == -1) {
        perror("stat failed");
        exit(EXIT_FAILURE);
    }

return statbuf.st_ino;
}

//入栈
int push(DirStack *s, const char *dirName) {
    if (s->top >= MAX_DEPTH - 1) {
        return -1;
    }
    s->top++;
    strcpy(s->data[s->top], dirName);
    return 0;
}
//出栈
int pop(DirStack *s, char *buf) {
    if (s->top == -1) {
        return -1;
    }
    //把栈里的名字，拷贝到 buf 指向的内存中
    strcpy(buf, s->data[s->top]);
    s->top--;
    return 0;
}
//判断栈是否为空
int isEmpty(DirStack *s) {
    return s->top == -1;
}

int find_name_in_parent(ino_t target_ino, char *name_buf) {
    DIR *dir = opendir(".");//打开当前目录
    //检查一下
    if (!dir) {
    perror("opendir failed");
    return 0;
    }
    struct dirent *entry;//此指针用于遍历目录文件
    struct stat statbuf;
    int found = 0;

    //遍历当前目录下的所有条目
    while ((entry = readdir(dir)) != NULL) {
        //跳过 “.” 和 “..”，避免死循环
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..")
== 0) {
            continue;
        }
        //对当前条目调用 stat() 获取其属性
        if(stat(entry->d_name, &statbuf) == -1){
             continue; //如果 stat 失败（如权限不足），跳过该条目
        }
        //如果该条目的Inode 等于我们要目标Inode
```

```
        if (statbuf.st_ino == target_ino) {
            strcpy(name_buf, entry->d_name);//将找到的名字存入缓冲区
            found = 1;
            break;//退出循环
        }
    }
    closedir(dir);//关闭目录流，防止内存泄漏
    return found;
}

int main(){
    DirStack stack;
    //初始化
    stack.top = -1;

    ino_t root_ino;
    struct stat root_stat;
    stat("/", &root_stat);
    root_ino = root_stat.st_ino;//获取根目录的Inode
    //获取当前目录的 inode
    ino_t cur_ino = get_inode(".");

    //如果当前已经是根目录，直接打印
    if (cur_ino == root_ino) {
        printf("/\n");
        return 0;
    }

    char buf[MAX_NAME_LEN];
    //逆推循环
    while (cur_ino != root_ino) {
         char dir_name[MAX_NAME_LEN];

        //切换到父目录并检查
        if (chdir("..") == -1) {
            perror("chdir failed");
            return 1;
        }
        //获取父目录的Inode
        ino_t parent_ino = get_inode(".");

        //在父目录中寻找cur_ino（即子目录的Inode）的名字
        if (find_name_in_parent(cur_ino, dir_name)) {
             // 找到名字后，压入栈中
            push(&stack, dir_name);
        } else {
            printf("Error: Cannot resolve path\n");
            return 1;
        }
        //更新当前Inode为父目录的Inode，继续下一轮循环
        cur_ino = parent_ino;
    }
    printf("/");
    while (!isEmpty(&stack)) {
        pop(&stack,buf);
        printf("%s", buf);
        //如果栈里还有元素，说明不是最后一级目录，需要加 "/"
        if (!isEmpty(&stack)) {
```

```
            printf("/");
        }
    }
    printf("\n");
    return 0;
}
```

最终境：算帝—— 递归与文件筛选

【任务 4.1：敢问，路在何方？】

其实怎么写要求里已经说的很详尽了，具体的操作也只是写了两个if，实现DFS，也就是将函数 find_file 放到 if 里面就完事儿了，不过在这里，我觉得我最好还是理一下函数的思路：

我们的目的是在一堆文件和文件夹里面找到文件 The_way_home.tar.gz ，找到的标准自然就是：strcmp(entry->d_name, target_name) == 0

下面重点阐明while具体是如何工作的：

1.跳过当前目录和父目录

2.声明了一个局部变量来存储 full_path，主要的作用就两个：一是作为可视的输出结果，二是作为递归的参数

3.获得Inode信息，并拷贝到statbuf中

4.若找到目标文件，立即输出

5.若现在的dir是目录（这里就用到statbuf了），则向下一层递归，并重复上述操作（当然还有之前的一系列操作）

6.当某一层目录遍历完仍未找到目标文件，当前层函数返回，回到上一层的 while 循环继续检查下一个条目——可能进入新的子目录递归，也可能继续遍历同级文件，如此反复，直至找到目标

```c
#include <stdio.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include <unistd.h>

// 递归查找指定文件，若找到则打印其相对路径
void find_file(const char *current_path, const char *target_name) {
    DIR *dir;
    struct dirent *entry;
    struct stat statbuf;

    if (!(dir = opendir(current_path))) {
        return; // 无法打开目录则跳过
    }

    while ((entry = readdir(dir)) != NULL) {
        // 忽略当前目录 . 和父目录 .. 防止无限死循环
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..")
== 0) {
            continue;
        }

        char full_path[1024];
```

```
        snprintf(full_path, sizeof(full_path), "%s/%s", current_path,
entry->d_name);

        if (stat(full_path, &statbuf) == -1) continue;

        // 请补全代码写在 TO DO 与 END OF TO DO 之间：
        // 要求：
        // 1. 检查当前 entry 的文件名是否与 target_name 匹配，若匹配则打印
full_path。
        // 2. 如果当前 entry 是一个目录，则需要对其进行深度优先的递归查找。
        // （提示：若(statbuf.st_mode & S_IFMT) == S_IFDIR则statbuf为目录文件）
        // TO DO
        if (strcmp(entry->d_name, target_name) == 0){
            printf("%s\n", full_path);
        }

        if((statbuf.st_mode & S_IFMT) == S_IFDIR){
            find_file(full_path, target_name);
        }

        // END OF TO DO
    }
    closedir(dir);
}
```

后记

在这之前，我其实有学习过一点关于Linux的内容，但大多数都是 “宏观” 层面的的操作，比如说一些实用的指令（包括处理文件的、管理用户的，甚至没有涉及代码程序的），各种系统中各种文件的职能等等，主要还是在学习 “怎么做”

这道题让我从另一个视角看到了不同的Linux：ls 是 opendir/readdir/stat 的组合，pwd 则是 chdir("..") + 按 inode 找名字 的逆推

说实话初次接触，信息量确实不小，据说初代Linux代码不到100行，不知道林纳斯是怎么写出来的（神仙），之前看恺翁老师的C语言课程时，其中有几个 ”my函数“系列 的视频也让我印象深刻

我觉得学习确实理应如此，不仅要知道 “怎么做” 还需要知道 “为什么”，正所谓：知其然，而知其所以然，我想自己写函数正是这样一个由浅入深的过程

做题过程中我还了解了硬链接和符号链接（软链接）的区别，不知道放在哪里比较好，索性就放在这儿了：

- 底层原理与 Inode
  - 硬链接：与原文件共享同一个 Inode，它们本质上是同一个文件数据的多个入口，在文件系统中没有实际的副本
  - 符号链接：是一个独立的特殊文件，拥有自己独立的 Inode。它的内容仅仅是存储了目标文件或目录的路径信息
- 对目录的支持
  - 硬链接：不能对目录创建硬链接（为了防止目录树出现循环引用导致文件系统崩溃）。
  - 符号链接：可以指向目录，也可以指向普通文件、设备文件等（有MAXSYMLINKS，不会无限递归）
- 跨文件系统能力
  - 硬链接：只能在同一个文件系统内创建，因为 Inode 编号只在当前文件系统内有意义
  - 符号链接：可以跨越不同的文件系统和磁盘分区进行链接

- 删除原文件的影响
  - 硬链接：删除原文件不会影响硬链接的访问，只要指向该 Inode 的硬链接数大于 0，文件的数据就会一直保留在磁盘上，只有当所有硬链接都被删除时，数据才会被真正清除
  - 符号链接：如果原文件被删除或移动，符号链接本身依然存在，但会失效成为“断链”，此时无法再正常访问数据
- 创建方式
  - 硬链接：使用 ln 命令创建，例如：ln 源文件 硬链接名
  - 符号链接：使用 ln -s 命令创建，例如：ln -s /绝对路径/源文件 链接名
- 目标不存在时的创建
  - 硬链接：必须指向一个已经存在的文件
  - 符号链接：可以指向一个当前并不存在的文件或路径

现在再看后记的内容，不禁莞尔，想到了上小学的我，每到周末就跑去图书馆看一下午的《斗破苍穹》，总是幻想能真的穿越到斗气大陆，开启一番别样的人生……

而现在，我仍然在图书馆，依然是一个阳光明媚的下午，甚至还迷恋着网文，总感觉什么都变了，什么都没变……

嗯，Niko学长也一定喜欢看网文吧，我最近正在看网文界最高的山——《诡秘之主》，有空一起交流一下（我可是十年老书虫）